

Daily Scholar Papers Report — 2026-07-09

2026-07-09

Daily Scholar Papers Report — 2026-07-09

[Download PDF](#)

Window covered: 2026-07-08 → 2026-07-09 (Google Scholar alerts + user-curated self-emails, last 24 h)

Executive Summary

A quiet Scholar-alert day — for the second run in a row the `scholaralerts-noreply@google.com` inflow was empty inside the 24-hour window, and a wider two-day probe confirms the last batch remains the 2026-07-06 one that was fully triaged in the 2026-07-07 report. The self-forwarded reading queue produced exactly one item: an IEEE Xplore stampPDF URL that resolves to **SFA-Miner** at S&P 2026, previously highlighted (abstract-only) in the 2026-07-07 digest. As a user-picked re-flag it is re-surfaced here with the confirmed S&P numbers from the CSDL abstract (181 API misuses, 1 CVE, four large open-source projects) and a slightly deeper structural framing. The paper still sits behind the IEEE paywall, so the analysis remains abstract-anchored.

Outstanding: 1 · **Keep:** 0 · **Borderline High-Priority:** 0

Highlighted Papers

#	Title	Authors	Venue	Link
1.1	SFA-Miner: Mining Path-Sensitive API Usage Patterns Via Symbolic Finite Automata	J. Jiang, M. Zheng, Q. Shi, X. Zhang	IEEE Symposium on Security and Privacy (S&P), 2026	IEEE · DOI

Outstanding

1.1 · SPECIFICATION-MINING · [USER-PICK] Path-sensitive API-usage patterns as symbolic finite automata — 181 misuses across Linux/OpenSSL/FFmpeg/httpd, 1 CVE — IEEE S&P 2026

SFA-Miner: Mining Path-Sensitive API Usage Patterns Via Symbolic Finite Automata J. Jiang, M. Zheng, Q. Shi, X. Zhang — IEEE Symposium on Security and Privacy, 2026 — [IEEE](#) · [DOI](#)

Re-highlighted at the reader's request. This paper first appeared in the 2026-07-07 digest as Outstanding 1.2, then on an abstract-only signal. Today's write-up updates that entry with the confirmed CSDL abstract from the S&P proceedings version (pages 3340–3357) and re-frames the contribution against the rest of the Qingkai Shi group's 2026 pipeline.

Problem framing. API misuses are a durable source of security-relevant defects, and mining “correct usage patterns” from code has a long lineage — PR-Miner, Jadet, GrouMiner, MUBench, and their descendants. The consistent weakness across a decade of this work is that mined patterns strip away path conditions: the miner learns “these calls tend to appear together” but not “these calls must appear together **when this precondition holds.**” Under-specified patterns push false positives into any downstream misuse detector; over-specified patterns miss entire usage families.

Approach. SFA-Miner represents each mined pattern as a *symbolic finite automaton* whose states are abstract program states and whose transitions carry predicates that mix API-call signatures with symbolic variables representing the calls' parameters. Distinct SFAs are mined per path condition, so the miner materialises the fact that the same API can have several legitimate usage patterns depending on the guarding context — for example, an initialiser that is required on the error path but forbidden on the success path shows up as two SFAs rather than one noisy one. Violations of the mined SFAs are flagged as candidate misuses.

Evaluation (confirmed). Applied to four widely used C code bases: **Linux kernel**, **OpenSSL**, **FFmpeg**, and **Apache httpd**. The abstract reports **181 API misuses** identified and **1 CVE** attributed. Per-project breakdown, precision/recall, and runtime cost require the paywalled full text.

Take. Path-sensitivity was the missing dimension in FSA-based specification mining, and publishing at S&P — a venue that rarely rewards mining papers without a strong offensive-security payoff — is the right signal for how much scaffolding this had to build. The SFA representation is also the plausible target IR for “conditional pattern” over LLM-agent tool invocations: swap “API-call transitions with predicates over parameters” for “tool-invocation transitions with predicates over tool arguments” and the semantic slot is identical. Worth revisiting once the S&P text is accessible.

Abstract-only limitations. Not yet known from the abstract: the predicate abstraction domain, the frequent-mining algorithm (Apriori-family over SFA fragments? closed-pattern mining? BIDE?), the false-positive rate on the misuse benchmark, or the SFA-construction procedure (predicate-first vs. cluster-first).

Cross-Paper Synthesis

A single-paper window makes cross-paper synthesis narrow by definition, but the SFA-Miner re-flag lets us tie a longer thread. Three papers from Qingkai Shi's group land at top venues in 2026 with the same underlying bet: *path-sensitive symbolic modelling is the backbone for the next generation of scalable static analysis*. SFA-Miner (S&P) mines usage patterns as symbolic automata; *Hermes* (OOPSLA'26) makes path-sensitive pointer analysis scalable enough to power sparse value-flow tools; *Sound and Precise Symbolic Automata Model for Stateful Software Systems* (CAV'26) provides the modelling foundations. Read together they sketch a stack: symbolic automata as the representation, path-sensitive pointer analysis as the reachability layer, and mined specifications as the “what should the program do” layer. Anyone building an API-misuse or protocol-conformance tool in the next 12 months should treat this trio as a joint reference. The near-term open question — and it is the one the paywalled SFA-Miner

text presumably answers — is how the SFA predicate domain is chosen, because that choice determines whether the mined specifications generalise across code bases or overfit to the training corpus.

Writing & Rationale Insights

Re-flagging a previously covered paper is a strong reader signal, and the correct pipeline response is not “already covered, skip” but “re-surface, and add whatever the new evidence justifies.” Today the new evidence is exactly two numbers (181 misuses, 1 CVE) plus the confirmation that the S&P proceedings version matches the pre-print announcement, so the write-up is deliberately incremental over the 2026-07-07 entry: it does not repeat the mining-history background at the same length, and it spends its new budget on the group-level portfolio framing that only becomes possible once you notice the three-venue pattern. Two more pipeline notes worth capturing. First: two consecutive zero-Scholar-alert days is not yet actionable — Scholar batches typically fire every 2–3 days, and a manual sanity check is only warranted after a third empty day. Second: when a paper is paywalled and no institutional access is available, the honest move is to write “abstract-only” plainly and list the open questions the abstract could not answer, rather than paper over the gap with plausible-sounding but ungrounded technical detail.